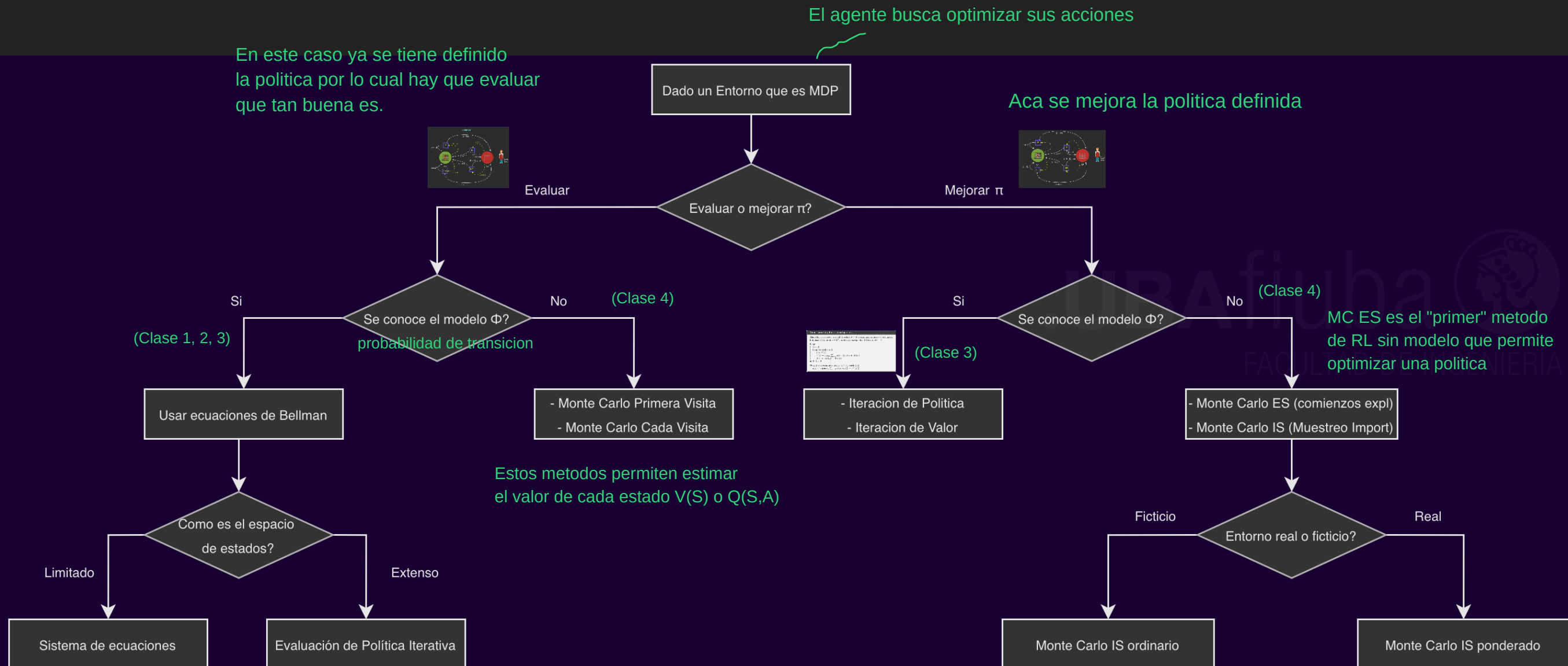


4

MÉTODOS DE MONTE CARLO

REPASO



Ej: si se tiene un gridworld de 5x5

REPASO

► El **valor de cada estado** de un MDP se puede calcular de varias formas:

✓ Sistema de N ecuaciones con N incógnitas.

$$v_{\pi}(A) = \frac{1}{4} [5 + 0.7v_{\pi}(B)] + \frac{1}{4} [0 + 0.7v_{\pi}(C)] + \frac{1}{2} [0 + 0.7v_{\pi}(A)]$$

$$v_{\pi}(B) = \frac{1}{4} [0 + 0.7v_{\pi}(A)] + \frac{1}{4} [0 + 0.7v_{\pi}(D)] + \frac{1}{2} [5 + 0.7v_{\pi}(B)]$$

✓ Algoritmo iterativo de Evaluación de Política.

✓ Métodos de Monte Carlo.

Con estos metodos se puede calcular el valor de estado o el valor de estado-accion

3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0

REPASO

► El **valor de cada estado** de un MDP se puede calcular de varias formas:

✓ Sistema de N ecuaciones con N incógnitas.

$$v_{\pi}(A) = \frac{1}{4} [5 + 0.7v_{\pi}(B)] + \frac{1}{4} [0 + 0.7v_{\pi}(C)] + \frac{1}{2} [0 + 0.7v_{\pi}(A)]$$

✓ Algoritmo iterativo de Evaluación de Política.

$$v_{\pi}(B) = \frac{1}{4} [0 + 0.7v_{\pi}(A)] + \frac{1}{4} [0 + 0.7v_{\pi}(D)] + \frac{1}{2} [5 + 0.7v_{\pi}(B)]$$

✓ Métodos de Monte Carlo.



INTRODUCCIÓN

- ▶ Supongamos que un entorno en donde cada celda es un **estado**.
- ▶ También supongamos que un **agente** interactúa con ese entorno y recibe una **recompensa** a medida que se desplaza aleatoriamente hacia su objetivo.

A -0.1 	B -0.1	C -0.1	D -0.1
E -0.1	F -0.1	G -0.1	H -0.1
I -0.1	J -0.1	K -0.1	L -0.1
M -0.1	N -5 	O -0.1	P +10 

INTRODUCCIÓN

- ▶ Si el valor de cada estado $v(s)$ se calculara promediando las recompensas que recibe el agente en diferentes episodios y su secuencia de acciones actual fuese "Abajo, Abajo, Abajo, Derecha" tendríamos (al final del episodio 1):

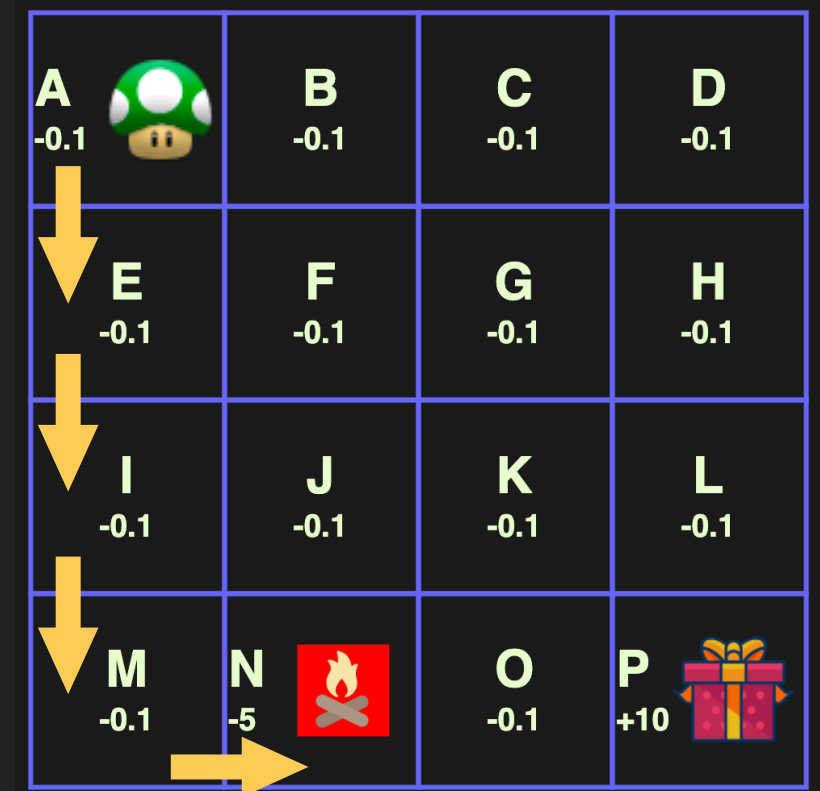
$v(N) = 0$ (porque es un estado absorbente).

$$v(M) = -5 + 0 = -5$$

$$v(I) = -0.1 + -5 = -5.1$$

...

Como no se conoce la probabilidad de transición, una forma de conocer los valores de estado es haciendo que el agente recorra la grilla y guarde la recompensa en cada episodio



INTRODUCCIÓN

- Para el episodio 2 tendríamos (si la secuencia fuese "Abajo, Abajo, Derecha, Abajo"):

$v(N) = 0$ (porque es un estado absorbente).

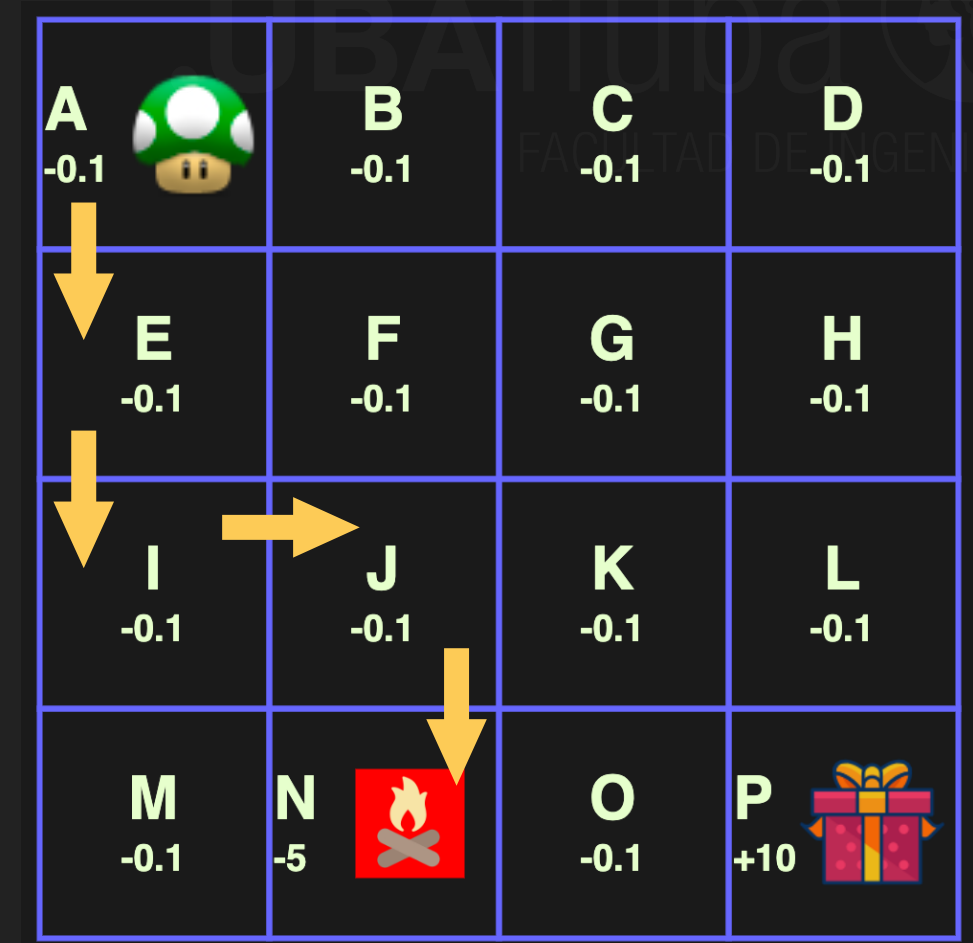
$$v(J) = -5 + 0 = -5$$

$$"v(I)" = -0.1 + -5 = -5.1$$

$$v(I) = (-5.1 + -5.1) / 2 = -5.1$$

Como ya se paso por el estado I, se hace un promedio

...



MÉTODO DE MONTE CARLO DE PRIMERA VISITA



INTRODUCCIÓN

- ▶ Las funciones de valor $v(s)$ y $q(s, a)$ que vimos hasta ahora se pueden estimar usando un enfoque distinto.
- ▶ En vez de usar la ecuación de Bellman podemos promediar las recompensas de cada episodio y tomar ese promedio como el valor $v(s)$ o $q(s, a)$.

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- ▶ Veamos esto con un ejemplo un poco mas completo:
- ▶ Supongamos un **buscador de tesoro** en un laberinto de 3x3.

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

► Estados: {A, B, C, D, E, F, G, H, I}

► Estado Inicial: Siempre empieza en A.

► Estados Terminales:

✓ I (Tesoro): Si llega a I, el episodio termina y recibe una recompensa grande (+10).

✓ F (Trampa): Si llega a F, el episodio termina y recibe una penalización (-5)

► Política (π) Fija (desde cualquier casilla no terminal):

✓ 60% de probabilidad de intentar moverse a la DERECHA. →

✓ 40% de probabilidad de intentar moverse ABAJO. ↓

✓ Si el movimiento elegido lo saca del laberinto (choca con un muro imaginario), se queda en la misma casilla para ese paso.

► Recompensas (R):

✓ Entrar en I: +10

✓ Entrar en F: -5

✓ Cualquier otro movimiento (o quedarse quieto por chocar): -0.1 (un pequeño costo por cada paso/tiempo).

► Factor de Descuento: $\gamma = 1$ (sin descuento, para simplificar). El valor es la suma total de recompensas futuras.

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

► Inicialización:

✓ $V(s) = 0$ para todos los estados $s \in \{A, B, C, D, E, G, H\}$. (F e I son terminales).

✓ $Returns(s) = []$ para todos los estados $s \in \{A, B, C, D, E, G, H\}$.

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

► Episodio 1:

- Empieza en **A**. La secuencia de movimientos aleatorios según la política (π) es: **Derecha, Abajo, Derecha**.

✓ **A** -> **B** ($R = -0.1$)

✓ **B** -> **E** ($R = -0.1$)

✓ **E** -> **F** (Trampa) ($R = -5$) --> Episodio Termina.

- Secuencia: **A**, **B**(-0.1), **E**(-0.1), **F**(-5)



A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA - EJEMPLO

► Cálculo de Retornos (G) Hacia Atrás ($\gamma=1$):

Para calcular el valor de estado, se empieza desde el ultimo estado

✓ G en F (terminal) = 0 (por definición)

✓ G en E (justo antes de F): $G = R(E \rightarrow F) + G \text{ en F} = -5 + 0 = -5$

✓ G en B (justo antes de E): $G = R(B \rightarrow E) + G \text{ en E} = -0.1 + (-5) = -5.1$

✓ G en A (justo antes de B): $G = R(A \rightarrow B) + G \text{ en B} = -0.1 + (-5.1) = -5.2$

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F  -5
G -0.1	H -0.1	I  +10

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

► Actualización de Valores de Estado (Solo Primera Visita):

✓ Estado E: ¿Primera visita en este episodio? Sí. Returns(E) = [-5]

$$V(E) = -5 / 1 = -5$$

✓ Estado B: ¿Primera visita en este episodio? Sí. Returns(B) = [-5.1]

$$V(B) = -5.1 / 1 = -5.1$$

✓ Estado A: ¿Primera visita en este episodio? Sí. Returns(A) = [-5.2]

$$V(A) = -5.2 / 1 = -5.2$$

► Valores después Ep. 1: V(A)=-5.2, V(B)=-5.1, V(E)=-5, otros=0

Si ya se visitó el estado, se promedia el valor según la cantidad de visitas que hubo en cada episodio

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

► Episodio 2:

► Empieza en A. Secuencia aleatoria según π : Abajo, Derecha, Derecha.

✓ A $\rightarrow$ D (R = -0.1)

✓ D $\rightarrow$ E (R = -0.1)

✓ E $\rightarrow$ F (Trampa) (R = -5) $\rightarrow$ Episodio Termina.

► Secuencia: A, D(-0.1), E(-0.1), F(-5)



A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

► Cálculo de Retornos (G) hacia atrás ($\gamma=1$):

✓ G en F = 0

✓ G en E = $R(E \rightarrow F) + G \text{ en F} = -5 + 0 = -5$

✓ G en D = $R(D \rightarrow E) + G \text{ en E} = -0.1 + (-5) = -5.1$

✓ G en A = $R(A \rightarrow D) + G \text{ en D} = -0.1 + (-5.1) = -5.2$

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

► Actualización de Valores de Estado (Solo Primera Visita):

✓ Estado E: ¿Primera visita en este episodio? Sí. Returns(E) = [-5, -5]

$$V(E) = (-5 + -5) / 2 = -5$$

✓ Estado D: ¿Primera visita en este episodio? Sí. Returns(D) = [-5.1]

$$V(D) = -5.1 / 1 = -5.1$$

Como ya se estuvo en el estado E y A, se promedia por 2

✓ Estado A: ¿Primera visita en este episodio? Sí. Returns(A) = [-5.2, -5.2]

$$V(A) = (-5.2 + -5.2) / 2 = -5.2$$

► Valores después Ep. 2: V(A)=-5.2, V(B)=-5.1, V(D)=-5.1, V(E)=-5, otros=0

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- ▶ **Episodio 3:**
- ▶ Empieza en A. Secuencia aleatoria según π : Derecha, Derecha, Abajo.
 - ✓ A $\rightarrow$ B ($R = -0.1$)
 - ✓ B $\rightarrow$ C ($R = -0.1$)
 - ✓ C $\rightarrow$ F (Trampa) ($R = -5$) $\rightarrow$ Episodio Termina.
- ▶ Secuencia: A, B(-0.1), C(-0.1), F(-5)



A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

► Cálculo de Retornos (G) hacia atrás ($\gamma=1$):

✓ G en C = -5

✓ G en B = -5.1

✓ G en A = -5.2

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

► Actualización de Valores (Solo Primera Visita):

✓ Estado C: ¿Primera visita en este episodio? Sí. Returns(C) = [-5]

$$V(C) = -5$$

✓ Estado B: ¿Primera visita en este episodio? Sí. Returns(B) = [-5.1, -5.1]

$$V(B) = -5.1$$

✓ Estado A: ¿Primera visita en este episodio? Sí. Returns(A) = [-5.2, -5.2, -5.2]

$$V(A) = -5.2$$

► Valores después Ep. 3: $V(A) = -5.2$, $V(B) = -5.1$, $V(C) = -5$, $V(D) = -5.1$, $V(E) = -5$, otros = 0

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

► Episodio 4:

► Empieza en A. Secuencia aleatoria según π : Abajo, Derecha, Abajo, Derecha.

✓ A $\rightarrow$ D (R = -0.1)

✓ D $\rightarrow$ E (R = -0.1)

✓ E $\rightarrow$ H (R = -0.1)

✓ H $\rightarrow$ I (Tesoro) (R = +10) $\rightarrow$ Episodio Termina.

► Secuencia: A, D(-0.1), E(-0.1), H(-0.1), I(+10)

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

► Cálculo de Retornos (G):

✓ G en I = 0

✓ G en H = $R(H \rightarrow I) + G \text{ en I} = 10 + 0 = 10$

✓ G en E = $R(E \rightarrow H) + G \text{ en H} = -0.1 + 10 = 9.9$

✓ G en D = $R(D \rightarrow E) + G \text{ en E} = -0.1 + 9.9 = 9.8$

✓ G en A = $R(A \rightarrow D) + G \text{ en D} = -0.1 + 9.8 = 9.7$

.UBAfiuba
FACULTAD DE INGENIERÍA

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

► Actualización de Valores (Solo Primera Visita):

✓ Estado H: ¿Primera visita en este episodio? Sí. Returns(H) = [10]

$$V(H) = 10$$

✓ Estado E: ¿Primera visita en este episodio? Sí. Returns(E) = [-5, -5, 9.9]

$$V(E) = (-5 -5 + 9.9) / 3 \approx -0.03$$

✓ Estado D: ¿Primera visita en este episodio? Sí. Returns(D) = [-5.1, 9.8]

$$V(D) = (-5.1 + 9.8) / 2 = 2.35$$

✓ Estado A: ¿Primera visita en este episodio? Sí. Returns(A) = [-5.2, -5.2, -5.2, 9.7]

$$V(A) = (-5.2*3 + 9.7) / 4 = -1.475$$

► Valores después Ep. 4: $V(A) \approx -1.48$, $V(B) = -5.1$, $V(C) = -5$, $V(D) = 2.35$, $V(E) \approx -0.03$, $V(H) = 10$, otros=0

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- ▶ Se continua generando episodios. Cada vez que se visita un estado por primera vez en un episodio, se añade su retorno calculado a la lista y se recalcula el valor promedio para ese estado.
- ▶ Si el agente, por ejemplo, en un episodio hiciera $A \rightarrow B \rightarrow E \rightarrow B \rightarrow C \rightarrow F$, la segunda visita a B no se usaría para actualizar $V(B)$. Solo se usaría el retorno calculado desde la primera vez que llegó a B en ese episodio.

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- ▶ Con suficientes episodios, los valores $V(s)$ convergerán a una estimación de lo bueno que es estar en cada casilla, dada la política de movimiento aleatoria y fija del agente.
- ▶ Sería posible ver que, estados como H (cerca del tesoro) tienden a tener valores altos, mientras que estados como C (cerca de la trampa) tienden a tener valores bajos.

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE PRIMERA VISITA – PSEUDOCODIGO

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

El limite del algoritmo es por la cantidad de episodios definido

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

MONTE CARLO – CONCEPTO

- ▶ El núcleo de los métodos de Monte Carlo es aprender promediando las recompensas que se obtienen durante muchos intentos, a los que llamamos "episodios".
- ▶ El punto es que el Agente solo necesita esa interacción, no necesita que alguien le diga las reglas del juego, las descubre por sí solo.
- ▶ Al igual que cuando usábamos los algoritmos de Iteración de Política e Iteración de Valor, con MC lo que se busca es averiguar lo que se llama la función de valor de estado $v(s)$.

MONTE CARLO – CONCEPTO

- ▶ Los métodos de Monte Carlo funcionan mediante muestreo aleatorio para obtener respuestas numéricas.
- ▶ Fueron desarrollados originalmente en la década de 1940 para tratar problemas matemáticos complejos que carecían de soluciones analíticas.
- ▶ Su nombre, "**Monte Carlo**", se eligió por la similitud entre este proceso basado en la aleatoriedad y los juegos de azar del famoso casino.



MONTE CARLO – CONCEPTO

- ▶ Existen dos enfoques principales de Monte Carlo para encontrar la función de valor de estado $v(s)$:

✓ MC de primera visita (first-visit MC) y,

✓ MC de cada visita (every-visit MC).

MONTE CARLO – CONCEPTO

- ▶ En **MC de primera visita**, para cada ejecución (cada episodio), por cada estado que ve, solo importa lo que sucede después de la primera vez que ve ese estado en ese episodio.
- ▶ De modo que se promedian las recompensas futuras de esa primera visita a través de todos los diferentes episodios.
- ▶ Si el agente se encuentra con ese mismo estado mas tarde en ese mismo episodio, simplemente lo ignora. Solo se enfoca en esa primera visita.
- ▶ El método de **MC de cada visita**, usa todas las visitas. Cada vez que un estado aparece en un episodio, registra la recompensa desde ese punto en adelante y la agrega al promedio para ese estado.
- ▶ El método de MC de primera visita es el más estudiado, eso es porque tiene algunas características interesantes, especialmente en lo que respecta a las estadísticas.
- ▶ Las muestras que se promedian son independientes, por lo que la **ley de los grandes números** entra en juego. Lo que eso significa es que el valor estimado **está garantizado a acercarse cada vez más al valor verdadero real cuanto más se visite ese estado**. Más visitas, mejor precisión.

MÉTODO DE MONTE CARLO DE CADA VISITA



MONTE CARLO DE CADA VISITA – EJEMPLO

- ▶ Estados: {A, B, C, D, E, F, G, H, I}
- ▶ Estado Inicial: Siempre empezamos en A.
- ▶ Estados Terminales: I, F.
- ▶ Política (π) Determinista: Desde cualquier casilla no terminal, el agente elige una de las cuatro direcciones (Arriba, Abajo, Izquierda, Derecha) con igual probabilidad (25% cada una). Si el movimiento elegido choca contra un muro (fuera del 3x3), el agente se queda en la misma casilla para ese paso.
- ▶ Recompensas (R): +10 (entrar I), -5 (entrar F), -0.1 (cualquier otro movimiento o quedarse quieto).
- ▶ Factor de Descuento (γ): 1.

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE CADA VISITA – EJEMPLO

► Inicialización:

✓ $V(s) = 0$ para $s \in \{A, B, C, D, E, G, H\}$.

✓ $Returns(s) = []$ para $s \in \{A, B, C, D, E, G, H\}$.

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE CADA VISITA – EJEMPLO

- ▶ **Episodio 1:** (Misma secuencia que antes, ahora es válida bajo la nueva política)
- ▶ Supongamos que la secuencia de movimientos aleatorios es: Derecha, Abajo, Derecha.
- ▶ Secuencia: A, B(-0.1), E(-0.1), F(-5) --> Termina

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE CADA VISITA – EJEMPLO

► Cálculo de Retornos (G) Hacia Atrás ($\gamma=1$):

✓ G en F = 0

✓ G en E = -5

✓ G en B = -5.1

✓ G en A = -5.2

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE CADA VISITA – EJEMPLO

- Actualización de Valores (Cada Visita): Recorremos (A, B, E).

✓ Visita E (t=2): Añadir -5 a Returns(E). $R(E)=[-5]$

$$V(E) = -5$$

✓ Visita B (t=1): Añadir -5.1 a Returns(B). $R(B)=[-5.1]$

$$V(B) = -5.1$$

✓ Visita A (t=0): Añadir -5.2 a Returns(A). $R(A)=[-5.2]$

$$V(A) = -5.2$$

- Valores después Ep. 1: $V(A)=-5.2$, $V(B)=-5.1$, $V(E)=-5$, otros=0

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE CADA VISITA – EJEMPLO

- ▶ **Episodio 2:** (Misma secuencia que antes)
- ▶ Supongamos secuencia: Abajo, Derecha, Derecha.
- ▶ Secuencia: A, D(-0.1), E(-0.1), F(-5) --> Termina

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE CADA VISITA – EJEMPLO

► Cálculo de Retornos (G):

✓ G en F = 0

✓ G en E = -5

✓ G en D = -5.1

✓ G en A = -5.2

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE CADA VISITA – EJEMPLO

- Actualización de Valores (Cada Visita): Recorremos (A, D, E).

✓ Visita E (t=2): Añadir -5 a R(E). R(E)=[-5, -5]

$$V(E) = (-5-5)/2 = -5$$

✓ Visita D (t=1): Añadir -5.1 a R(D). R(D)=[-5.1]

$$V(D) = -5.1$$

✓ Visita A (t=0): Añadir -5.2 a R(A). R(A)=[-5.2, -5.2]

$$V(A) = (-5.2-5.2)/2 = -5.2$$

- Valores después Ep. 2: V(A)=-5.2, V(B)=-5.1, V(D)=-5.1, V(E)=-5, otros=0.

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE CADA VISITA – EJEMPLO

- ▶ **Episodio 3** (revisita):
- ▶ Supongamos secuencia: Derecha, Abajo, Arriba, Derecha, Abajo.
- ▶ Secuencia: A, B(-0.1), E(-0.1), B(-0.1), C(-0.1), F(-5) --> Termina

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE CADA VISITA – EJEMPLO

► Cálculo de Retornos (G):

✓ G en F = 0

✓ G en C (antes de F): $-5 + 0 = -5$

✓ G en B (antes de C, segunda visita a B, $t=3$): $-0.1 + (-5) = -5.1$

✓ G en E (antes de B, $t=2$): $-0.1 + (-5.1) = -5.2$

✓ G en B (antes de E, primera visita a B, $t=1$): $-0.1 + (-5.2) = -5.3$

✓ G en A (antes de B, $t=0$): $-0.1 + (-5.3) = -5.4$

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE CADA VISITA – EJEMPLO

► Actualización de Valores (Cada Visita): Recorremos (A, B, E, B, C).

✓ Estado Actual (antes Ep 3): $V(A)=-5.2$, $V(B)=-5.1$, $V(D)=-5.1$, $V(E)=-5$, $V(C)=0$, $V(H)=0$

► Actualizaciones:

✓ Visita C (t=4): Añadir $G=-5$ a $R(C)$. $R(C)=[-5]$. $V(C) = -5$.

✓ Visita B (t=3, 2da visita): Añadir $G=-5.1$ a $R(B)$. $R(B)$ era $[-5.1]$, ahora $[-5.1, -5.1]$. $V(B) = \text{Promedio}([-5.1, -5.1]) = -5.1$.

✓ Visita E (t=2): Añadir $G=-5.2$ a $R(E)$. $R(E)$ era $[-5, -5]$, ahora $[-5, -5, -5.2]$. $V(E) = \text{Promedio}([-5, -5, -5.2]) = -15.2 / 3 \approx -5.07$.

✓ Visita B (t=1, 1ra visita): Añadir $G=-5.3$ a $R(B)$. $R(B)$ era $[-5.1, -5.1]$, ahora $[-5.1, -5.1, -5.3]$. $V(B) = \text{Promedio}([-5.1, -5.1, -5.3]) = -15.5 / 3 \approx -5.17$.

✓ Visita A (t=0): Añadir $G=-5.4$ a $R(A)$. $R(A)$ era $[-5.2, -5.2]$, ahora $[-5.2, -5.2, -5.4]$. $V(A) = \text{Promedio}([-5.2, -5.2, -5.4]) = -15.8 / 3 \approx -5.27$.

► Valores después Ep. 3: $V(A)\approx-5.27$, $V(B)\approx-5.17$, $V(C)=-5$, $V(D)=-5.1$, $V(E)\approx-5.07$, $V(H)=0$. (La revisita a B afectó los promedios de B, E y A).

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE CADA VISITA – EJEMPLO

- ▶ **Episodio 4:** (Misma secuencia que antes, llegando al tesoro)
- ▶ Supongamos secuencia: Abajo, Derecha, Abajo, Derecha.
- ▶ Secuencia: A, D(-0.1), E(-0.1), H(-0.1), I(+10) --> Termina

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE CADA VISITA – EJEMPLO

► Cálculo de Retornos (G):

✓ G en I = 0

✓ G en H = 10

✓ G en E = 9.9

✓ G en D = 9.8

✓ G en A = 9.7

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE CADA VISITA – EJEMPLO

- ▶ Actualización de Valores (Cada Visita): Recorremos (A, D, E, H).
 - ✓ Estado Actual (antes Ep 4): $V(A) \approx -5.27$, $V(B) \approx -5.17$, $V(C) = -5$, $V(D) = -5.1$, $V(E) \approx -5.07$, $V(H) = 0$
- ▶ Actualizaciones:
 - ✓ Visita H (t=3): Añadir $G=10$ a $R(H)$. $R(H)=[10]$. $V(H) = 10$.
 - ✓ Visita E (t=2): Añadir $G=9.9$ a $R(E)$. $R(E)$ era $[-5, -5, -5.2]$, ahora $[-5, -5, -5.2, 9.9]$. $V(E) = \text{Promedio}([-5, -5, -5.2, 9.9]) = -5.3 / 4 = -1.325$.
 - ✓ Visita D (t=1): Añadir $G=9.8$ a $R(D)$. $R(D)$ era $[-5.1]$, ahora $[-5.1, 9.8]$. $V(D) = \text{Promedio}([-5.1, 9.8]) = 4.7 / 2 = 2.35$.
 - ✓ Visita A (t=0): Añadir $G=9.7$ a $R(A)$. $R(A)$ era $[-5.2, -5.2, -5.4]$, ahora $[-5.2, -5.2, -5.4, 9.7]$. $V(A) = \text{Promedio}([-5.2, -5.2, -5.4, 9.7]) = -6.1 / 4 = -1.525$.

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

MONTE CARLO DE CADA VISITA – EJEMPLO

► Valores Finales después de 4 Episodios:

✓ $V(A) \approx -1.53$

✓ $V(B) \approx -5.17$

✓ $V(C) = -5$

✓ $V(D) = 2.35$

✓ $V(E) \approx -1.33$

✓ $V(G) = 0$ (No visitado)

✓ $V(H) = 10$

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -5 
G -0.1	H -0.1	I +10 

Si luego de entrenar al agente no se visitó un estado, y si se tiene la posibilidad de empezar desde cualquier estado, entonces se inicia desde los estados no visitados para obtener el valor de todos los estados. Esto se hace así porque ese estado puede generar una mejor recompensa.

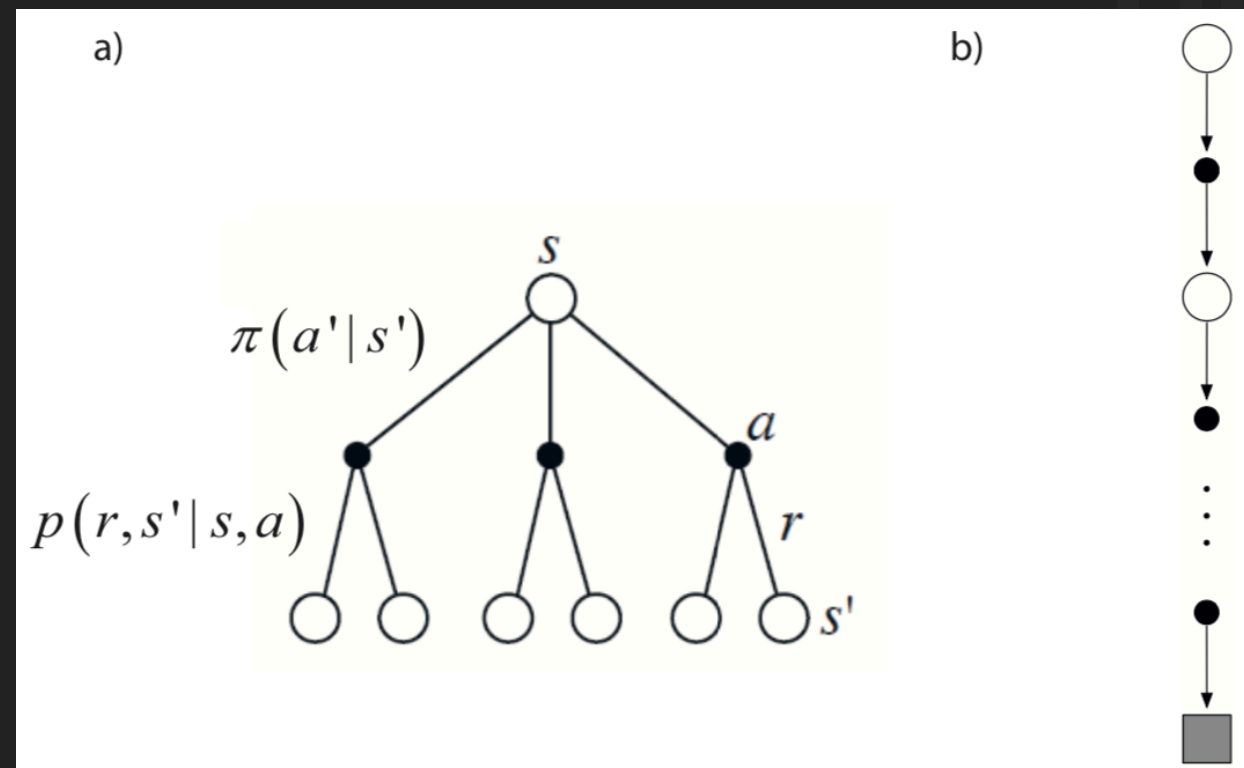
MONTE CARLO DE CADA VISITA – EJEMPLO

- ▶ Al permitir el movimiento hacia arriba, la secuencia del Episodio 3 que contenía la revisita (E $\rightarrow$ B) se vuelve válida y natural bajo la política.
- ▶ El algoritmo MC de Cada Visita procesa esta secuencia utilizando ambas visitas a B para actualizar $V(B)$, $V(E)$ y $V(A)$ con los retornos calculados desde cada uno de esos puntos.
- ▶ Después de 4 episodios, empezamos a ver cómo los valores reflejan la experiencia: **H (cercano al tesoro) tiene un valor alto**, **C (cercano a la trampa) tiene un valor bajo**, y los estados intermedios como D y E empiezan a mostrar valores positivos debido a la posibilidad de alcanzar el tesoro. Aún se necesitan muchos más episodios para que los valores converjan de manera fiable.

A -0.1		B -0.1	C -0.1
D -0.1	E -0.1	F -5	
G -0.1	H -0.1	I +10	

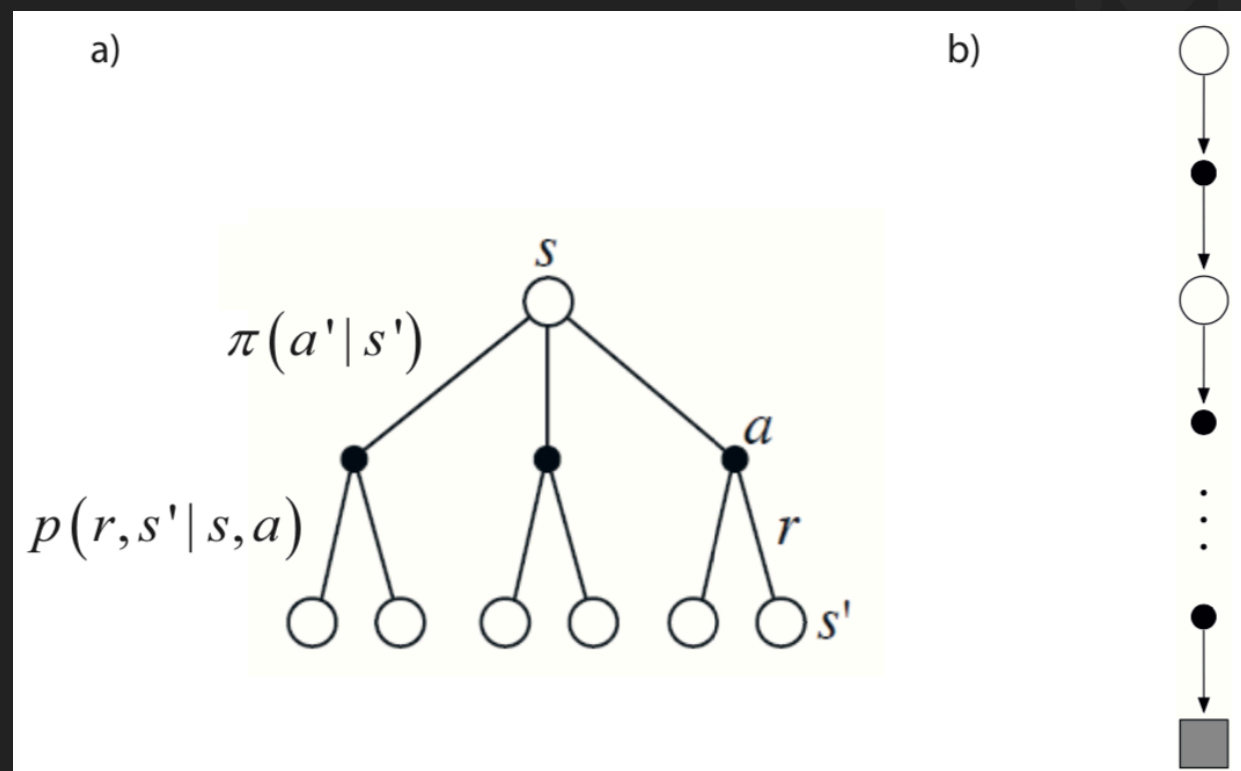
DIFERENCIAS ENTRE MC Y PD

- Los diagramas de respaldo de PD (Programación Dinámica) mostraban cada posible siguiente paso que se podía dar desde un estado, junto con las probabilidades de que ocurriera cada uno y el valor de cada estado resultante.



DIFERENCIAS ENTRE MC Y PD

- ▶ El diagrama de respaldo de Monte Carlo muestra la trayectoria real, la secuencia específica de estados, acciones y recompensas que realmente ocurren en un episodio simulado.
figura b)
- ▶ En lugar de mostrar cada posible siguiente paso, solo muestra la única trayectoria que se tomó.
- ▶ Analogía: Es como mostrar la ruta real en un mapa en lugar de todas las posibles rutas que podría haberse tomado.



DIFERENCIAS ENTRE MC Y PD

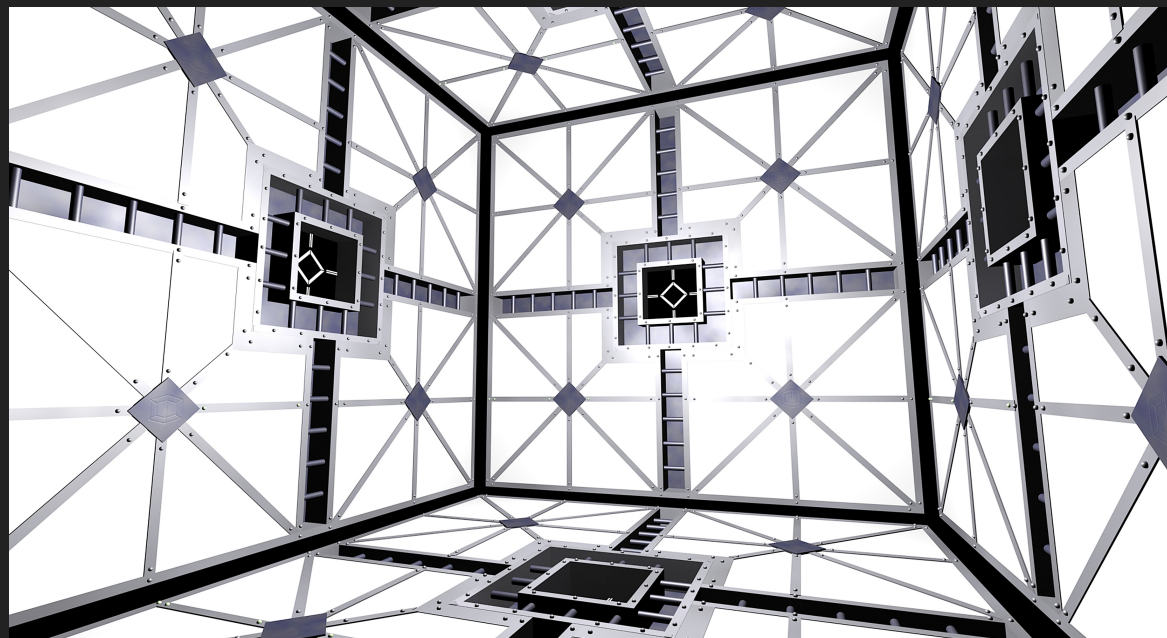
- ▶ Monte Carlo no hace "bootstrap". ya que el valor que obtiene es el real.
- ▶ El 'bootstrapping' en este contexto significa usar las estimaciones actuales de otros estados para estimar el valor del estado que se está analizando.

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')],$$

- ▶ La programación dinámica hace exactamente eso, es decir que constantemente actualiza los valores basándose en lo que cree que vale el siguiente estado.
- ▶ Monte Carlo no hace eso, solo mira las recompensas reales que obtuvo después de estar en ese estado. No se basa en ninguna suposición sobre otros estados.
- ▶ Es una forma más directa de averiguar qué tan bueno es un estado, simplemente basándose en lo que realmente sucedió después de estar allí.
- ▶ Se ejecutan una gran cantidad de simulaciones que pasen por esos estados y no tiene en cuenta el resto.

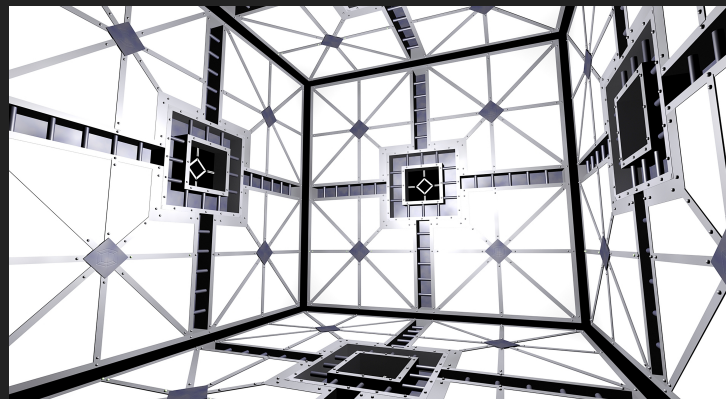
ESTIMACIÓN DE LOS VALORES DE ACCIÓN $Q(S, A)$

- ▶ Si no se tiene un modelo, significa que **no se sabe** qué va a pasar a continuación o **qué recompensas** se obtendrá.
- ▶ Saber qué tan bueno es un estado no es suficiente. También se necesita saber **qué tan bueno es tomar acciones específicas en ese estado**, es decir, conocer los valores de acción $q(s, a)$ y luego seguir la política desde allí.
- ▶ Analogía: Es como saber lo bueno que es estar en un cierto lugar versus saber lo bueno que es abrir una puerta específica en ese lugar.
- ▶ Lo que realmente se busca es averiguar la mejor acción a tomar en cada posible situación. Eso se llama la función de valor de acción óptima.



ESTIMACIÓN DE LOS VALORES DE ACCIÓN $Q(S, A)$

- ▶ ¿cómo usar Monte Carlo para estimar estos valores de acción?
- ▶ En lugar de solo considerar **estados**, se consideran **pares estado-acción**. Esto es, ¿se visitó un estado y luego se tomó una acción en particular? Ese es un par estado-acción.
- ▶ Para esto se pueden usar los mismos enfoques, MC de **primera visita** y de **cada visita**, para estimar el valor de estos pares. Luego **se promedian las recompensas** que se obtienen después de tomar esas acciones.
- ▶ Analogía: Es como entrar en una habitación y luego abrir una puerta específica, se busca saber, en promedio, qué tan buen resultado se obtiene después de hacer eso.
- ▶ En este punto surge un problema. ¿Qué pasa si nunca se prueban ciertas acciones en ciertos estados, especialmente si la política siempre dice que haga lo mismo?
- ▶ Si nunca se explora se podría perder obtener un resultado mejor. Es análogo a que si siempre se abre la misma puerta en una habitación, con lo cual podría nunca encontrar el mejor lugar.
- ▶ Se necesita probar alternativas diferentes para descubrir qué es realmente lo mejor. Para abordar esto lo más lógico es hacer: **Comienzos Exploratorios (Exploring Starts)**.

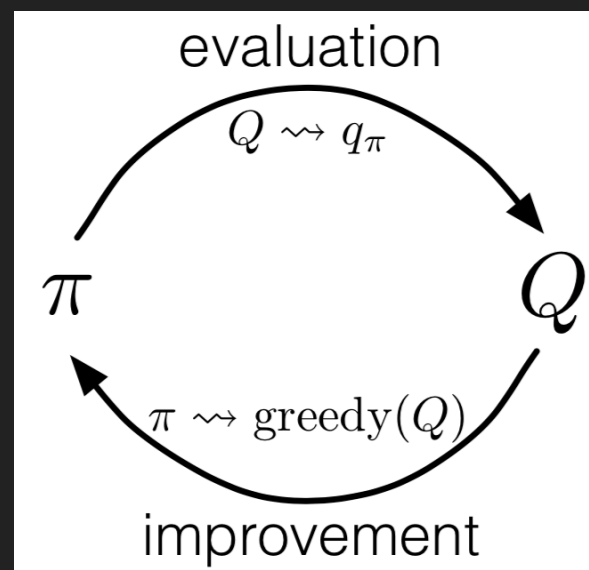


CONTROL MONTE CARLO



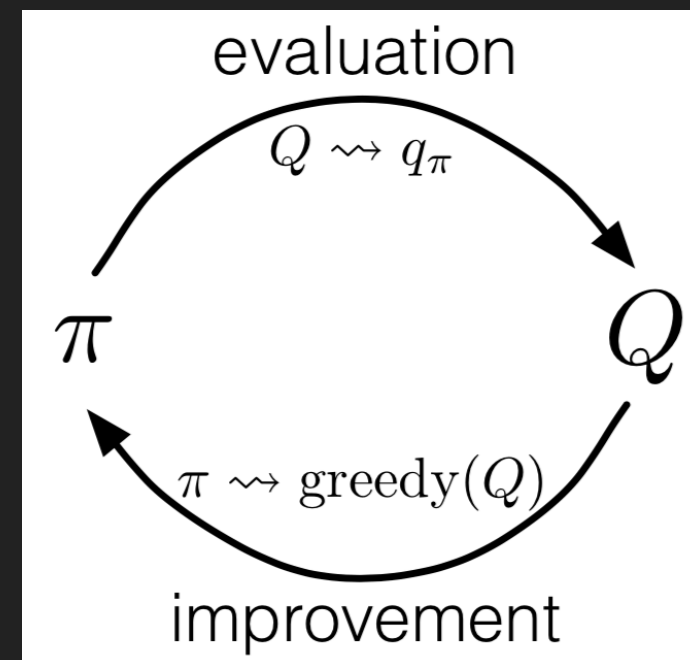
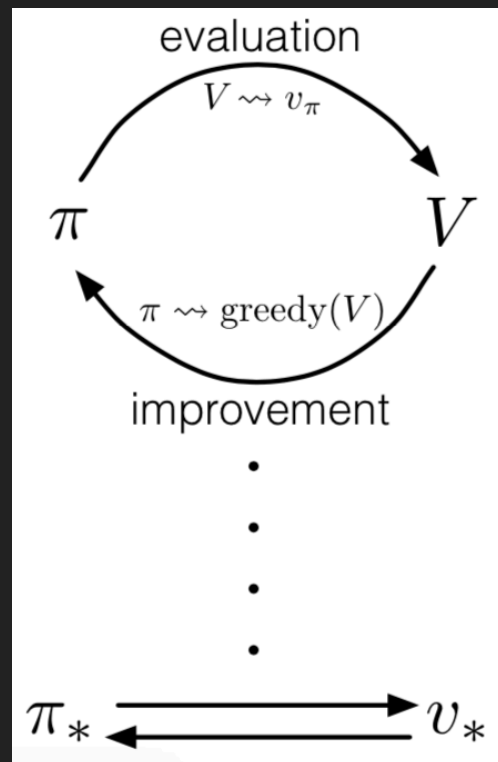
CONTROL MONTE CARLO

- ▶ "Control" = "Optimización"
- ▶ Control en RL = Encontrar una política óptima para maximizar la recompensa.
- ▶ ¿Porque se denomina "Control"?
- ▶ El agente al elegir acciones, está "controlando" su trayectoria a través de los estados para maximizar la recompensa.



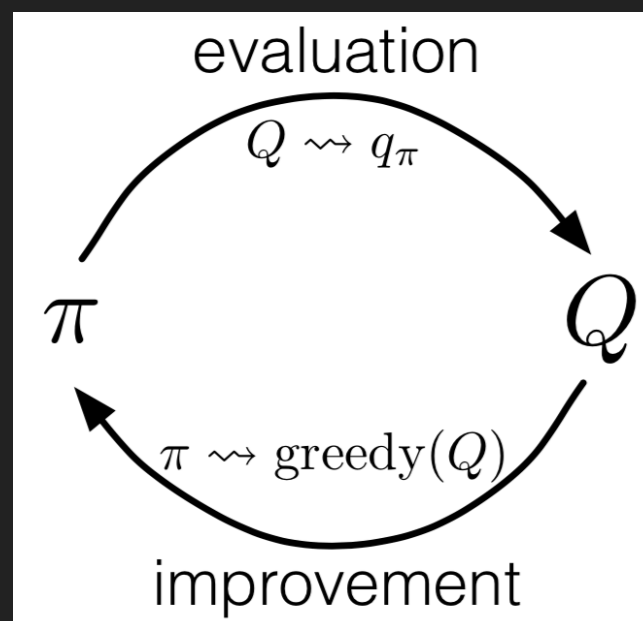
CONTROL MONTE CARLO

- ¿Cómo pasamos de averiguar qué tan bueno es un estado a encontrar realmente la mejor manera de actuar?
- Se usa la idea de **Iteración de Política Generalizada (GPI)**, es decir, el ciclo de evaluar la estrategia actual y luego mejorarla basándose en lo que se aprende.



CONTROL MONTE CARLO

- ▶ En PD se usaba el conocimiento del entorno para averiguar qué tan buena era la política, luego se implementaba una política "greedy" (voraz o codiciosa), lo que significa que siempre elegía la acción que parecía ser mejor.
- ▶ Con el Control Monte Carlo se aprende qué tan buenas son las acciones probándolas realmente y viendo qué sucede. Luego se mejora la política eligiendo las acciones que parecen llevar a los mejores resultados y para conseguir esto se usa **Comienzos Exploratorios.**



COMIENZOS EXPLORATORIOS (EXPLORING STARTS)



COMIENZOS EXPLORATORIOS (EXPLORING STARTS)

- ▶ Significa que se asume que cada episodio comienza en un par estado-acción aleatorio.
- ▶ Cada posible punto de partida es igualmente probable.
- ▶ Esto asegura que, eventualmente, probaremos cada acción en cada estado.
- ▶ Es una forma de garantizar que estamos explorando todo.

A -0.1	B -0.1	C -0.1
D -0.1	E -0.1	F -0.1 
G -0.1  Inicio	H -0.1	I +10 

COMIENZOS EXPLORATORIOS (EXPLORING STARTS)

► Monte Carlo ES realiza lo siguiente:

- ✓ Comienza eligiendo aleatoriamente un **estado** y **acción de inicio**, asegurándose de que cada posible punto de partida tenga una oportunidad, esa es la parte de **Comienzos Exploratorios**. **1**
- ✓ Luego deja que el agente juegue un episodio completo siguiendo su política actual. **2**
- ✓ Una vez que termina el episodio, **revisa cada paso**, y para cada par **estado-acción**, calcula la recompensa total que obtuvo desde ese punto en adelante. Usa esto para actualizar su estimación de **cuán buena es esa acción en ese estado**. **3**
- ✓ Finalmente, actualiza la política misma. En este punto es **más probable** que elija las acciones que parecen ser las mejores hasta ahora. **4**

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
 $Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
 $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

Loop forever (for each episode):

1 Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0

2 Generate an episode from S_0, A_0 , following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$
 $G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$: **(empieza desde el ultimo estado)**

3 $G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

4 $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$

La cantidad de episodios
y los pasos son parametros
que se definen

COMIENZOS EXPLORATORIOS (EXPLORING STARTS)

- ▶ **Objetivo:** Encontrar la política óptima (la mejor forma de moverse), no solo evaluar una política fija.
- ▶ **Valores de Acción (Q):** MC ES utiliza $Q(s, a)$, el valor esperado de tomar la acción 'a' en el estado 's' y luego seguir la política. Se necesita esto para decidir la mejor acción sin conocer el modelo.
- ▶ **Exploring Starts (ES):** Cada episodio **DEBE** comenzar en un par estado-acción (s, a) elegido al azar. Esto asegura que eventualmente se prueben todas las acciones en todos los estados.
- ▶ **Iteración Política Generalizada (GPI):** Se alterna entre:
 - ✓ **Evaluar** $Q(s, a)$ basado en los episodios generados (usando la política actual).
 - ✓ **Mejorar** la política haciéndola greedy (codiciosa) respecto a los $Q(s, a)$ actuales.
- ▶ **First-Visit (implícito):** Generalmente, para un par (s, a) dado, solo se usa el retorno de la **primera vez que aparece** en un episodio para actualizar su $Q(s, a)$.

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
 $Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
 $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

Loop forever (for each episode):

Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0
 Generate an episode from S_0, A_0 , following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$
 $G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$
 Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:
 Append G to $Returns(S_t, A_t)$
 $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
 $\pi(S_t) \leftarrow \text{argmax}_a Q(S_t, a)$

CONTROL MONTE CARLO – EJEMPLO

- ▶ Entorno: Laberinto 3x3.
- ▶ Estados Terminales: **I** (Tesoro), **F** (Trampa).
- ▶ Acciones: **UP, DOWN, LEFT, RIGHT** (necesarias para ES). Chocar = quedarse.
- ▶ Recompensas (R): **+10** (entrar I), **-5** (entrar F), **-0.1** (otros pasos/quedarse).
- ▶ Factor de Descuento (γ): **1**.

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -0.1 
G -0.1	H -0.1	I +10 

CONTROL MONTE CARLO – EJEMPLO

► Inicialización:

► $Q(s, a)$: Inicializa $Q(s, a) = 0$ para todos los pares estado-acción (s no terminal, $a \in \text{Acciones}$).

► **Política $\pi(s)$** Inicializa una política arbitraria (y determinista a la vez):

✓ $\pi(A) = \text{'RIGHT'}$

✓ $\pi(B) = \text{'RIGHT'}$

✓ $\pi(C) = \text{'DOWN'}$

✓ $\pi(D) = \text{'RIGHT'}$

✓ $\pi(E) = \text{'DOWN'}$

✓ $\pi(F)$ es terminal

✓ $\pi(G) = \text{'UP'}$

✓ $\pi(H) = \text{'RIGHT'}$

✓ $\pi(I)$ es terminal

► $\text{Returns}(s, a)$: Inicializa listas vacías para acumular los retornos de la primera visita para cada par (s, a) .

A -0.1 	B -0.1	C -0.1
D -0.1	E -0.1	F -0.1 
G -0.1	H -0.1	I +10 

CONTROL MONTE CARLO – EJEMPLO

► Episodio 1:

► Exploring Start:

✓ Estado inicial al azar: **E**.

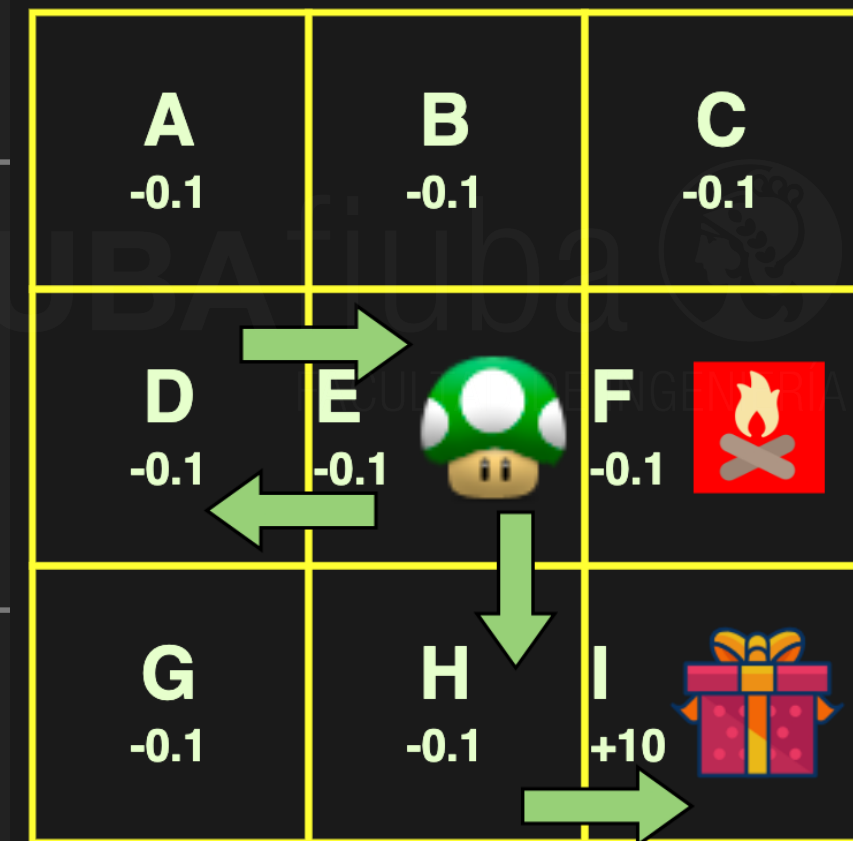
✓ Primera acción al azar: **LEFT**.

Siempre el estado inicial y la primer accion es aleatoria (paso 1 del algoritmo). Luego se sigue la politica definida



CONTROL MONTE CARLO – EJEMPLO

- ▶ **Episodio 1:**
- ▶ Generar Resto del Episodio (siguiendo π inicial):
- ▶ Agente en E, ejecuta LEFT. Va a D ($R = -0.1$).
- ▶ Agente en D. **Consulta política:** $\pi(D) = \text{'RIGHT'}$. Ejecuta RIGHT. Va a E ($R = -0.1$).
- ▶ Agente en E. **Consulta política:** $\pi(E) = \text{'DOWN'}$. Ejecuta DOWN. Va a H ($R = -0.1$).
- ▶ Agente en H. **Consulta política:** $\pi(H) = \text{'RIGHT'}$. Ejecuta RIGHT. Va a I (Tesoro) ($R = +10$). --> Termina.
- ▶ Secuencia: $(S_0=E, A_0=LEFT, R_1=-0.1), (S_1=D, A_1=RIGHT, R_2=-0.1), (S_2=E, A_2=DOWN, R_3=-0.1), (S_3=H, A_3=RIGHT, R_4=+10)$



CONTROL MONTE CARLO – EJEMPLO

► Episodio 1:

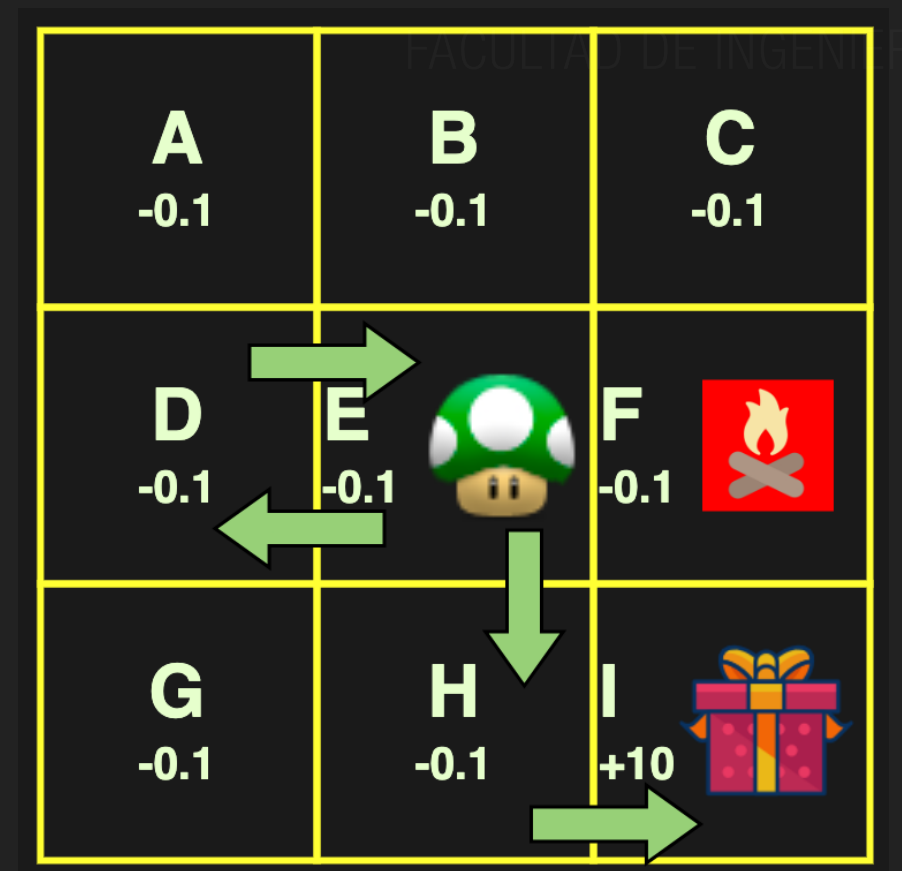
► Calcular Retornos (G) Hacia Atrás ($\gamma=1$):

✓ G para (H, RIGHT) [t=3]: $R_4 + 0 = 10$

✓ G para (E, DOWN) [t=2]: $R_3 + G_3 = -0.1 + 10 = 9.9$

✓ G para (D, RIGHT) [t=1]: $R_2 + G_2 = -0.1 + 9.9 = 9.8$

✓ G para (E, LEFT) [t=0]: $R_1 + G_1 = -0.1 + 9.8 = 9.7$



CONTROL MONTE CARLO – EJEMPLO

► Episodio 1:

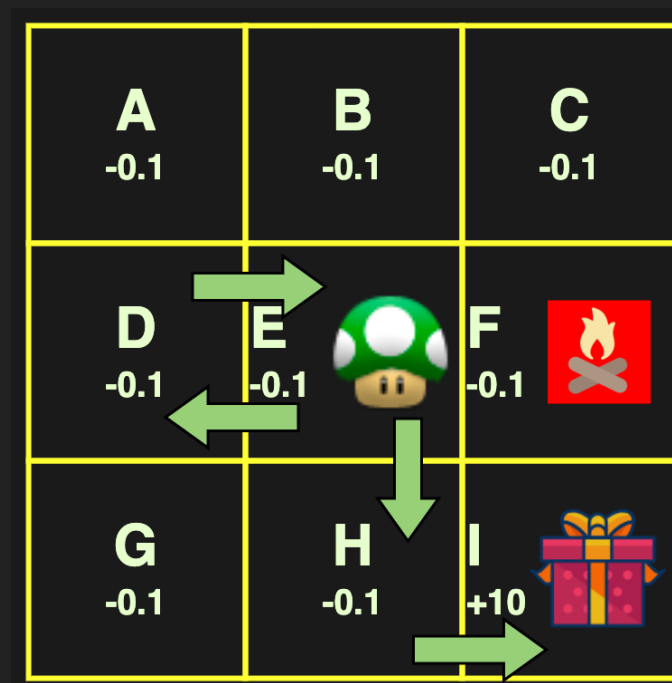
► Actualizar $Q(s, a)$ (First-Visit del par (s, a)):

✓ Par (H, RIGHT): 1ª visita. $\text{Returns}[(H, 'RIGHT')].\text{append}(10)$. $Q(H, \text{RIGHT}) = 10$.

✓ Par (E, DOWN): 1ª visita. $\text{Returns}[(E, 'DOWN')].\text{append}(9.9)$. $Q(E, \text{DOWN}) = 9.9$.

✓ Par (D, RIGHT): 1ª visita. $\text{Returns}[(D, 'RIGHT')].\text{append}(9.8)$. $Q(D, \text{RIGHT}) = 9.8$.

✓ Par (E, LEFT): 1ª visita. $\text{Returns}[(E, 'LEFT')].\text{append}(9.7)$. $Q(E, \text{LEFT}) = 9.7$.



CONTROL MONTE CARLO – EJEMPLO

► Episodio 1:

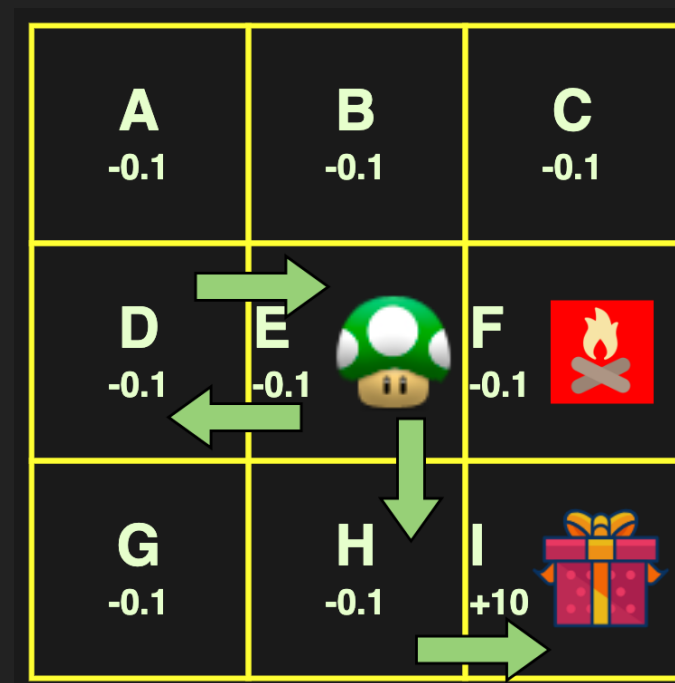
► Mejorar Política $\pi(s)$ (Greedy para estados visitados):

✓ Estado H: $Q(H, U/D/L)=0$, $Q(H, R)=10$. Mejor='RIGHT'. $\pi(H) = \text{'RIGHT'}$ (Confirma).

✓ Estado E: $Q(E, U/R)=0$, $Q(E, L)=9.7$, $Q(E, D)=9.9$. Mejor='DOWN'. $\pi(E) = \text{'DOWN'}$ (Confirma).

✓ Estado D: $Q(D, U/D/L)=0$, $Q(D, R)=9.8$. Mejor='RIGHT'. $\pi(D) = \text{'RIGHT'}$ (Confirma).

✓ La política para A, B, C, G no cambia aún.



CONTROL MONTE CARLO – EJEMPLO

► Episodio 2:

En cada episodio se hace inicio exploratorio!!

► Exploring Start:

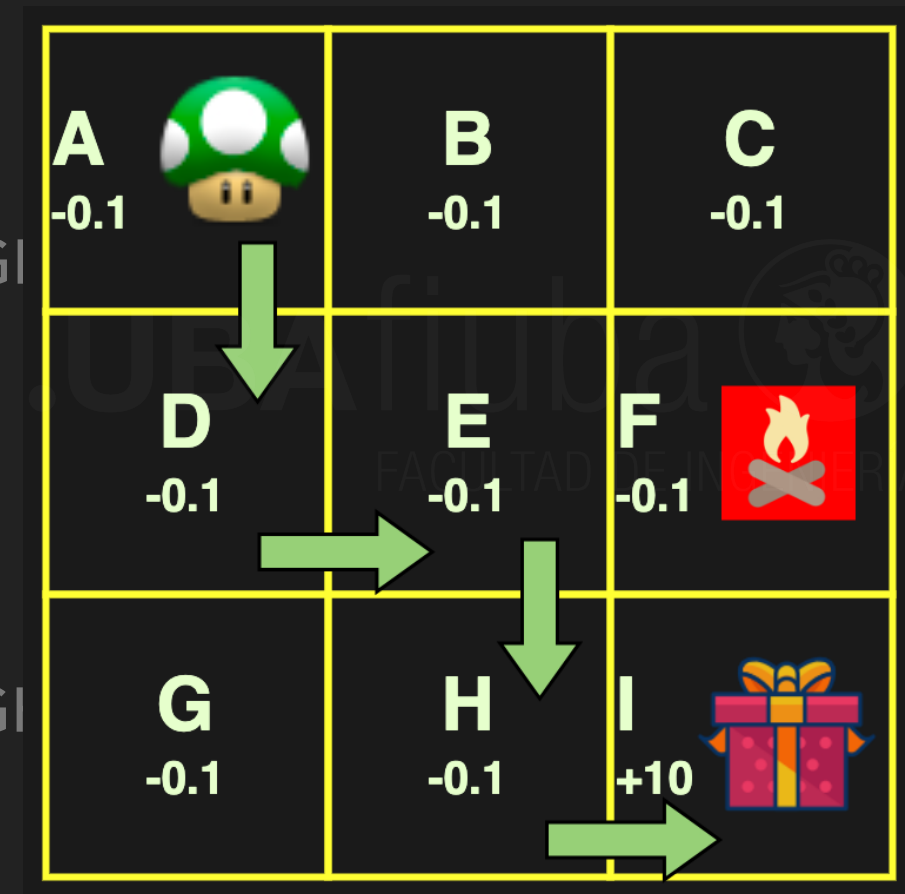
✓ Estado inicial al azar: **A**.

✓ Primera acción al azar: **DOWN**.

A -0.1 	B -0.1	C -0.1
 D -0.1	E -0.1	F -0.1 
G -0.1	H -0.1	I +10 

CONTROL MONTE CARLO – EJEMPLO

- ▶ **Episodio 2:**
- ▶ Generar Resto del Episodio (siguiendo π actualizada):
- ▶ Agente en A, ejecuta DOWN. Va a D ($R = -0.1$).
- ▶ Agente en D. **Consulta política:** $\pi(D) = \text{'RIGHT'}$. Ejecuta RIGHT. Va a E ($R = -0.1$).
- ▶ Agente en E. **Consulta política:** $\pi(E) = \text{'DOWN'}$. Ejecuta DOWN. Va a H ($R = -0.1$).
- ▶ Agente en H. **Consulta política:** $\pi(H) = \text{'RIGHT'}$. Ejecuta RIGHT. Va a I (Tesoro) ($R = +10$). → Termina.
- ▶ Secuencia: $(S_0=A, A_0=DOWN, R_1=-0.1)$, $(S_1=D, A_1=RIGHT, R_2=-0.1)$, $(S_2=E, A_2=DOWN, R_3=-0.1)$, $(S_3=H, A_3=RIGHT, R_4=+10)$



CONTROL MONTE CARLO – EJEMPLO

► Episodio 2:

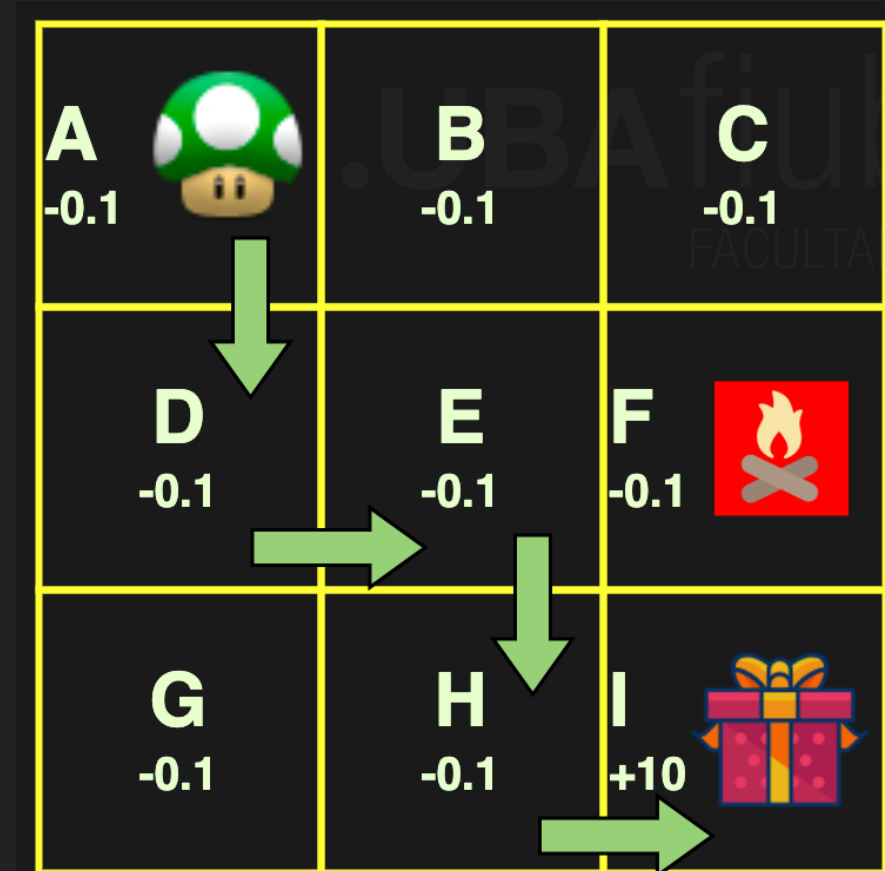
► Calcular Retornos (G): (Mismos cálculos que en Ep. 1, pero aplicados a estos pares)

✓ G para (H, RIGHT) [t=3]: **10**

✓ G para (E, DOWN) [t=2]: **9.9**

✓ G para (D, RIGHT) [t=1]: **9.8**

✓ G para (A, DOWN) [t=0]: **9.7**



CONTROL MONTE CARLO – EJEMPLO

► Episodio 2:

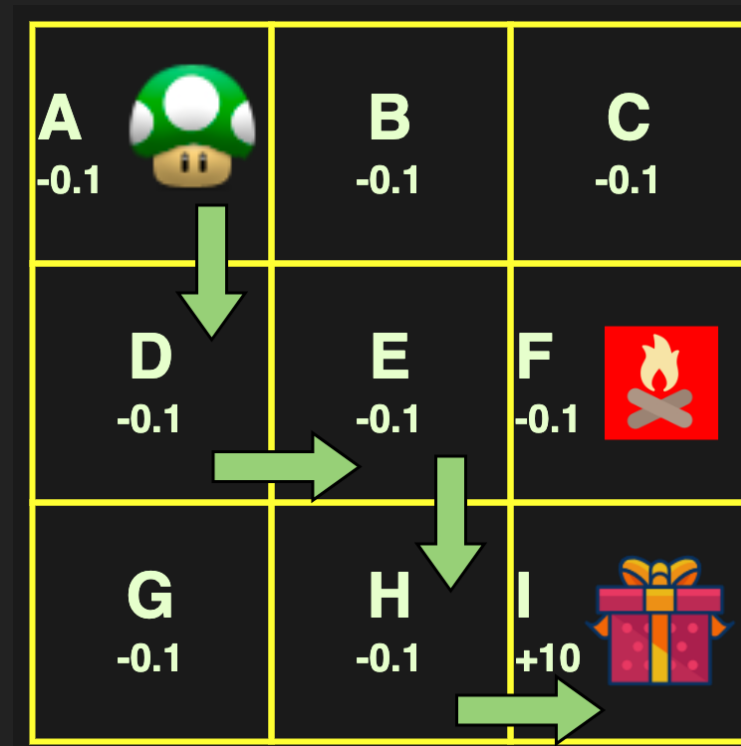
► Actualizar $Q(s, a)$ (First-Visit del par):

✓ Par (H, RIGHT): 1ª visita en este episodio. Returns[(H, 'RIGHT')] era [10], ahora [10, 10]. $Q(H, \text{RIGHT}) = 10$.

✓ Par (E, DOWN): 1ª visita en este episodio. Returns[(E, 'DOWN')] era [9.9], ahora [9.9, 9.9]. $Q(E, \text{DOWN}) = 9.9$.

✓ Par (D, RIGHT): 1ª visita en este episodio. Returns[(D, 'RIGHT')] era [9.8], ahora [9.8, 9.8]. $Q(D, \text{RIGHT}) = 9.8$.

✓ Par (A, DOWN): 1ª visita. Returns[(A, 'DOWN')].append(9.7). $Q(A, \text{DOWN}) = 9.7$.

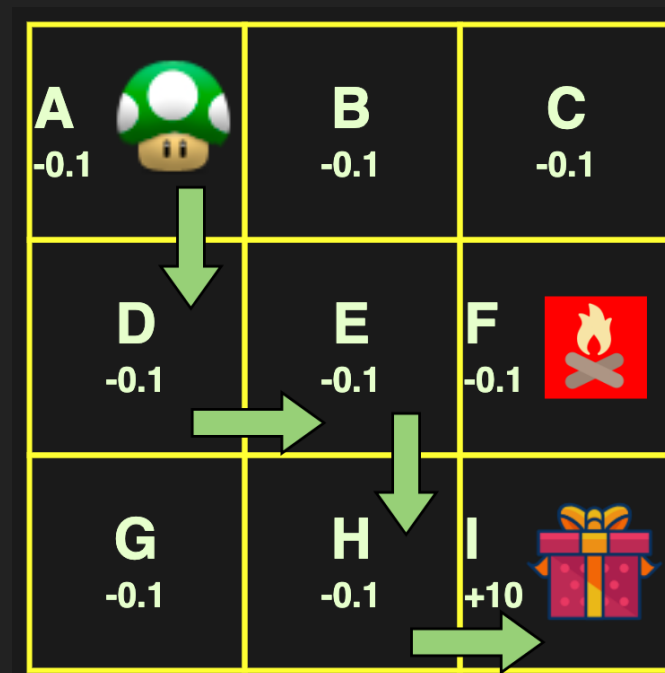


CONTROL MONTE CARLO – EJEMPLO

► Episodio 2:

► Mejorar Política $\pi(s)$:

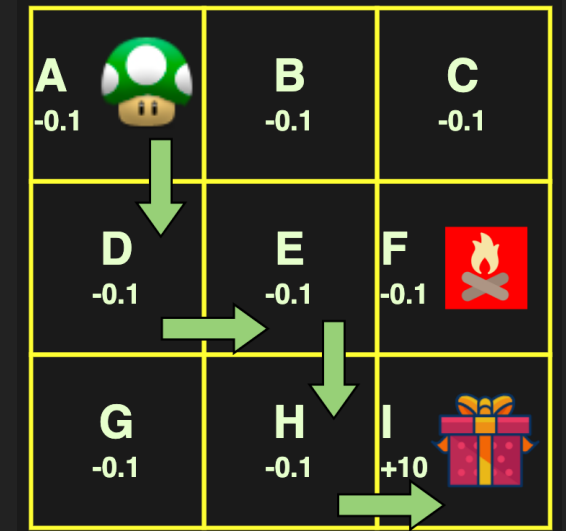
- ✓ Estado H: $Q(H, R)=10$ (máx). $\pi(H) = \text{'RIGHT'}$.
- ✓ Estado E: $Q(E, D)=9.9$ (máx). $\pi(E) = \text{'DOWN'}$.
- ✓ Estado D: $Q(D, R)=9.8$ (máx). $\pi(D) = \text{'RIGHT'}$. (Confirma).
- ✓ Estado A: $Q(A, L/U/R)=0$, $Q(A, D)=9.7$ (máx). $\pi(A) = \text{'DOWN'}$. (Cambió).



Cambio la política del estado A (optimizo)

CONTROL MONTE CARLO – EJEMPLO

- ▶ Continuar iterando...
- ▶ Se sigue este proceso. Es decir, con cada episodio:
 - ✓ Se explora un nuevo par (s, a) al inicio.
 - ✓ Se sigue la política actual (que es greedy respecto a los Q actuales).
 - ✓ Se calculan los retornos G .
 - ✓ Se actualiza el valor Q del primer par (s', a') encontrado en el episodio que coincide con uno ya visto, usando G .
 - ✓ Se actualiza la política para los estados visitados, haciéndola greedy respecto a los Q recién actualizados.
- ▶ Eventualmente, $Q(s, a)$ se acerca a $Q^*(s, a)$ y $\pi(s)$ se acerca a $\pi^*(s)$.
- ▶ Los valores de acciones cercanos al tesoro (como $Q(H, \text{RIGHT})$) serán altos, y los cercanos a la trampa (como $Q(C, \text{DOWN})$, $Q(E, \text{RIGHT})$) serán bajos.



ALGORITMOS ON-POLICY VS. OFF-POLICY

ALGORITMOS ON-POLICY VS OFF-POLICY

- ▶ Monte Carlo ES siempre comienza con un estado-acción aleatorio.

A -0.1	B -0.1	C -0.1
D -0.1	E -0.1	F -0.1 
G -0.1  Inicio	H -0.1	I +10 

ALGORITMOS ON-POLICY VS OFF-POLICY

► ¿Cuándo usar Monte Carlo ES?

Entorno	¿Permite Exploring Starts?	Motivo
Péndulo invertido	Sí	Si se discretiza el espacio de estados (redondeo de la posición y velocidad de la barra)
Tic Tac Toe	Sí	Espacio pequeño, estados/acciones fácilmente controlables
Blackjack (simplificado)	Sí	Se puede forzar cualquier par estado-acción inicial
Gridworld (mundos de cuadrícula)	Sí	Se puede colocar al agente en cualquier celda con cualquier acción
Nim / Pig Dice Game	Sí	Estados discretos y manipulables
Racetrack (Sutton & Barto)	Sí	Estados discretizados y controlables
Connect Four (versión pequeña)	No	No se puede si una columna está llena.
Ajedrez, Go, Shogi	No	Estados complejos, no todos alcanzables segun las reglas de juego
Póker / Juegos con estados ocultos	No	No se conoce el estado completo, imposible establecer pares estado-acción arbitrarios
Dominó (simplificado)	No	No se puede iniciar una partida con cualquier ficha en cualquier lugar: las jugadas deben respetar el valor de conexión.

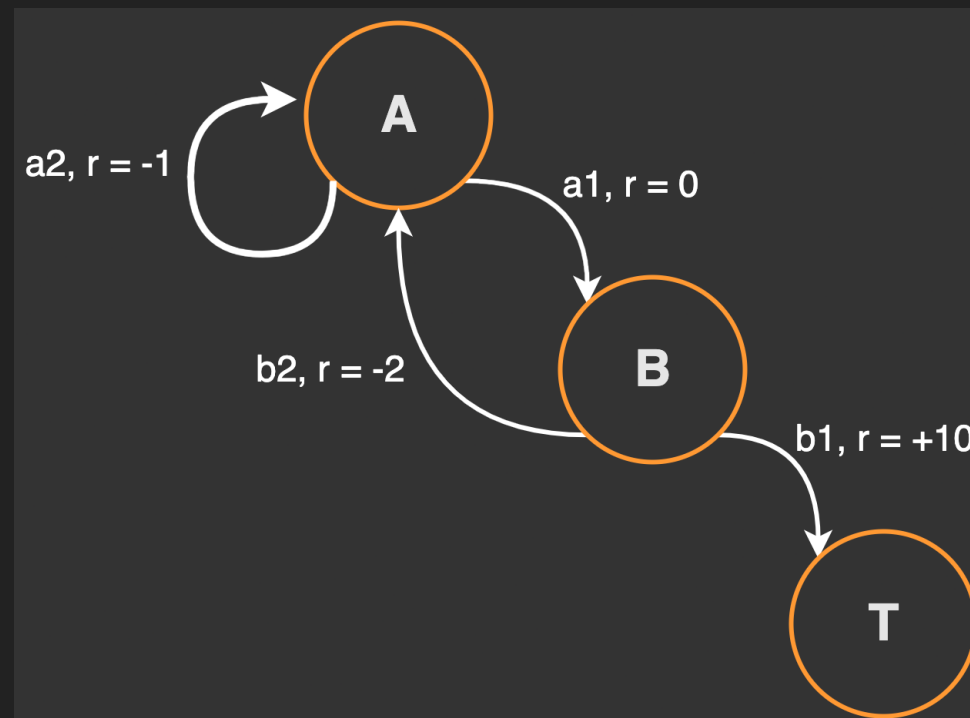
ALGORITMOS ON-POLICY VS OFF-POLICY

- ▶ ¿Qué sucede cuando no podemos garantizar que el agente comience en cada posible **par estado-acción**?
- ▶ Cuando esto ocurre se utilizan 2 métodos o enfoques:
 - ✓ **On-policy** (en la política)
 - ✓ **Off-policy** (fuera de la política)



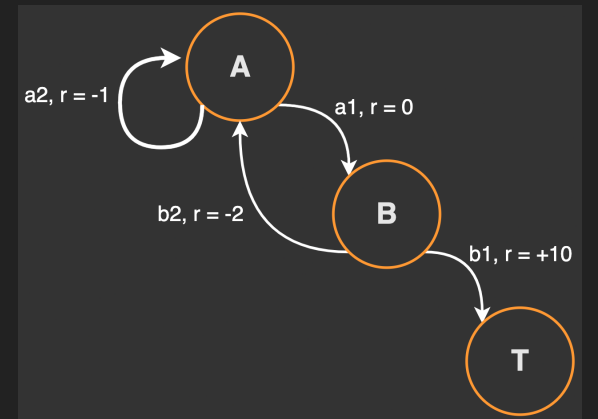
EJEMPLO

- ▶ Política Objetivo (π - Target Policy): La que se quiere aprender y optimizar.
 - ✓ Desde A: Siempre toma acción a1. (Probabilidad de $a1 = 1.0$, Probabilidad de $a2 = 0.0$)
 - ✓ Desde B: Siempre toma acción b1. (Probabilidad de $b1 = 1.0$, Probabilidad de $b2 = 0.0$)
- ▶ Política de Comportamiento (b - Behavior Policy): La que usa el agente para explorar (aleatoria equiprobable).
 - ✓ Desde A: Toma acción a1 con probabilidad 0.5 , toma acción a2 con probabilidad 0.5 .
 - ✓ Desde B: Toma acción b1 con probabilidad 0.5 , acción b2 con probabilidad 0.5 .



EJEMPLO

- ▶ Generando Datos con la Política de Comportamiento (b)
 - ✓ Episodio 1: $A \rightarrow B \rightarrow T$ (Acciones: $a1, b1$). Retorno G desde $A = 10$.
 - ✓ Episodio 2: $A \rightarrow A \rightarrow B \rightarrow T$ (Acciones: $a2, a1, b1$). Retorno G desde $A = 9$.
 - ✓ Episodio 3: $A \rightarrow B \rightarrow A \rightarrow B \rightarrow T$ (Acciones: $a1, b2, a1, b1$). Retorno G desde $A = 8$.
- ▶ El problema es que los datos que tenemos no nos dicen directamente qué habría pasado si hubiéramos estado siguiendo la política objetivo.
- ▶ Observamos los retornos 10, 9 y 8. Si los promediamos (≈ 9), ¿refleja eso el valor de seguir la política objetivo π ?
- ▶ No, porque los episodios 2 y 3 se generaron tomando acciones (como la acción $a2$ desde A en el episodio 2, o la acción $b2$ desde B en el episodio 3) que la política objetivo π nunca realizaría.
- ▶ La política π siempre tomaría $a1$ desde A y $b1$ desde B . Los resultados 9 y 8 están influenciados por la exploración aleatoria de la política de comportamiento b .

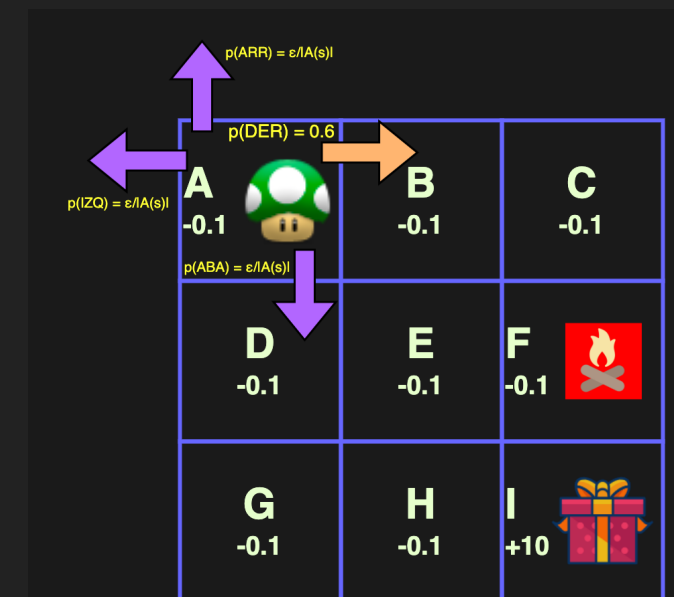
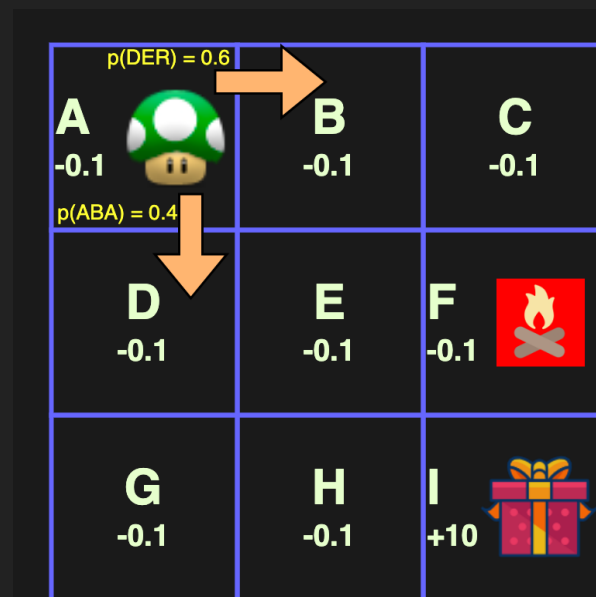


ALGORITMOS ON-POLICY VS OFF-POLICY

- ▶ Los métodos **on-policy** aprenden sobre la política que el agente está usando realmente (Ej. Monte Carlo ES).
- ▶ Los métodos **off-policy** pueden aprender sobre una política diferente incluso mientras el agente está haciendo otra cosa.
- ▶ Analogía: Si uno quisiera aprender manejar un coche tendría 2 opciones:
 - ✓ **on-policy**: Es como animarse y hacer. Aprendo sobre la marcha (autodidacta)
 - ✓ **off-policy**: Es observar a alguien más que ya domina el manejo del coche.

ALGORITMOS ON-POLICY VS OFF-POLICY

- ▶ Para resolver el problema de los **comienzos exploratorios** la técnica mas común es usar políticas suaves (**soft policies**).
- ▶ **Soft policies** → en cada estado la política tiene al menos una pequeña probabilidad de elegir cualquiera de las acciones.
- ▶ Ya que no hay **ES**, la propia política de comportamiento debe incluir un componente de **exploración**.
- ▶ Cuando **Soft policies** estima que una acción es la mejor aun así prueba otras acciones de vez en cuando para asegurarse de que no se está perdiendo algo mejor.
- ▶ Una forma común de hacer esto es con algo llamado **política épsilon-voraz** (**epsilon-greedy**).



POLÍTICA EPSILON-GREEDY

- ▶ Con una probabilidad de $1-\epsilon$, el agente elige la acción que cree que es mejor. Pero luego, con una pequeña probabilidad ϵ , simplemente elige una acción aleatoria.
- ▶ ϵ -greedy en gran medida está usando su conocimiento, pero también se asegura de explorar un poco.
- ▶ En lugar de hacer la política completamente greedy, la hacemos ϵ -greedy.
- ▶ $1-\epsilon+\epsilon/|\mathcal{A}(s)|$, es la probabilidad de que la acción greedy sea seleccionada.

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(s)| & \text{si } a = \arg \max_a Q(s,a) \\ \epsilon/|\mathcal{A}(s)| & \text{si } a \neq \arg \max_a Q(s,a) \end{cases}$$

POLÍTICA EPSILON-GREEDY

- ▶ $1-\epsilon+\epsilon/|A(s)|$, es la probabilidad de que la acción greedy sea seleccionada.
- ▶ ejemplo: $|A(s)| = 4$ y $\epsilon = 0.1$
 - ✓ Hay una acción greedy (la mejor)
 - ✓ Hay 3 acciones no-greedy
- ▶ Cada acción no-greedy se elegirá con probabilidad $\epsilon/|A(s)|=0.1/4 = 0.025$
- ▶ (Las 3 acciones no-greedy suman: $3 \times 0.025 = 0.075$)
- ▶ La acción greedy se elegirá con probabilidad $1-\epsilon+\epsilon/|A(s)| = 1-0.1+0.1/4 = 0.925$
- ▶ (la suma de todas las probabilidades es $0.925+0.075 = 1.0$)

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(s)| & \text{si } a = \arg \max_a Q(s,a) \\ \epsilon/|\mathcal{A}(s)| & \text{si } a \neq \arg \max_a Q(s,a) \end{cases}$$

GLIE

- ▶ Es el concepto de reducir gradualmente ϵ a 0.
- ▶ Permite que el agente:
 - ✓ **Explore** suficiente al principio (cuando ϵ es grande) para descubrir buenos valores Q .
 - ✓ **Explota** más el conocimiento adquirido posteriormente (cuando ϵ es pequeño) para converger hacia la política óptima.
- ▶ Se busca que ϵ tienda a 0 a medida que el tiempo/episodios tienden a infinito.
- ▶ Una forma de implementarlo es hacer $\epsilon_k = 1 / k$, donde k es el número de episodio. A medida que k aumenta, ϵ_k se acerca a 0.
- ▶ **GLIE** significa "Greedy in the Limit with Infinite Exploration" (Codicioso en el Límite con Exploración Infinita).
- ▶ Garantiza que el algoritmo converja a la política óptima (π^*).



MÉTODO OFF-POLICY

- ▶ ¿Por qué querríamos alguna vez **aprender** sobre una política mientras el agente se **comporta** realmente de acuerdo con una completamente diferente?
- ▶ Una ventaja en hacer eso es que se puede **aprender** sobre la política óptima, la que es realmente la mejor, mientras el agente se **comporta** de una manera que le permite explorar más.
- ▶ De modo que se puede estar **probando acciones** diferentes mientras se sigue **aprendiendo** sobre lo que es verdaderamente mejor.
- ▶ Con el aprendizaje **off-policy**, se puede aprender de datos que fueron generados por **otra persona** o incluso por un **experto humano**. No tiene porque hacer toda la exploración el agente mismo en todos los casos.

MÉTODO OFF-POLICY

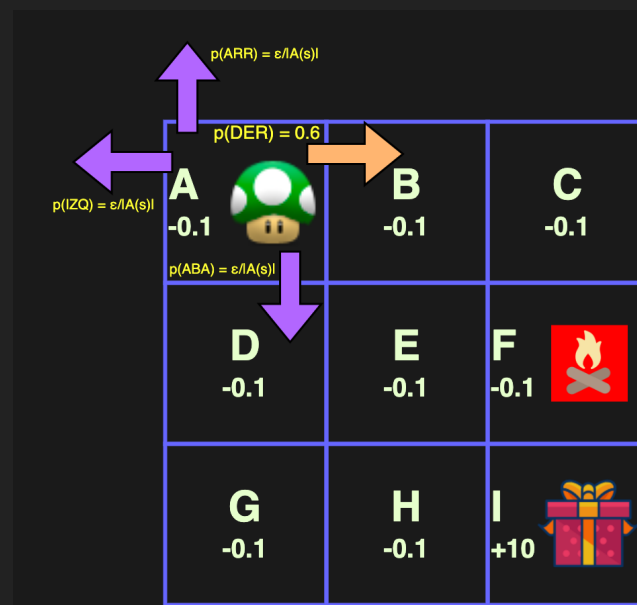
- ▶ Política objetivo (target policy - π), la que se quiere **aprender**.
- ▶ Política de comportamiento (behavior policy - μ o b), es lo que el agente **hace** para explorar.

MUESTREO POR IMPORTANCIA (IMPORTANCE SAMPLING – IS)

FACULTAD DE INGENIERÍA

MUESTREO POR IMPORTANCIA (IMPORTANCE SAMPLING - IS)

- ▶ El problema es que los **G** o **Returns[]** que tenemos no nos dicen directamente qué habría pasado si hubiéramos estado siguiendo la **política objetivo**.
- ▶ Para resolver esto se usa el **muestreo por importancia** (importance sampling). Es una forma de usar los datos que tenemos, aunque sean de una política diferente, para averiguar qué habría pasado si estuviéramos siguiendo la política que realmente nos interesa.
- ▶ **IS** es una técnica matemática que permite **ponderar** los resultados (los **retornos G**) obtenidos bajo la **política de comportamiento** (μ) para que reflejen mejor lo que habría sucedido bajo la **política objetivo** (π^*).



MUESTREO POR IMPORTANCIA (IMPORTANCE SAMPLING - IS)

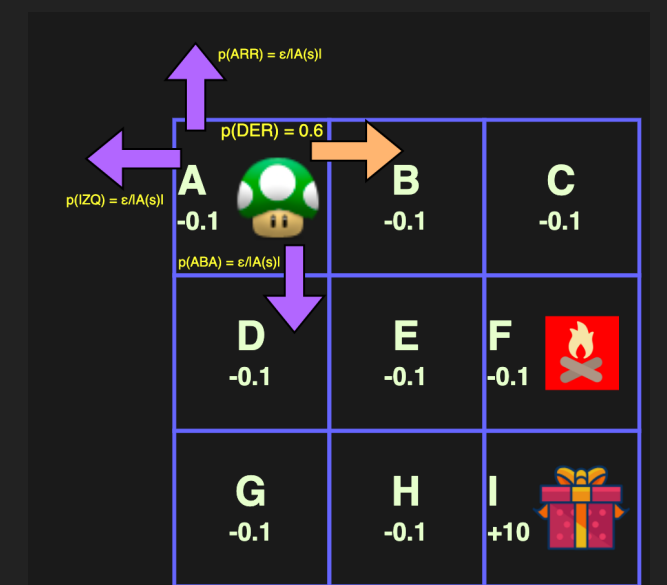
- Muestreo por importancia **ordinario**

$$\rho = \prod_{t=0}^{T-1} \frac{\pi(a_t|s_t)}{b(a_t|s_t)} \quad V(s) \leftarrow \text{promedio de } \rho \cdot G$$

- Muestreo por importancia **ponderado**.

$$\rho = \prod_{t=0}^{T-1} \frac{\pi(a_t|s_t)}{b(a_t|s_t)} \quad V(s) \leftarrow \frac{\sum \rho \cdot G}{\sum \rho}$$

- Donde **G** es el retorno total desde s_t



MUESTREO POR IMPORTANCIA (IMPORTANCE SAMPLING – IS)

► Muestreo por importancia ordinario (pseudocódigo)

Para cada episodio:

Generar episodio: $(s_0, a_0, r_1, s_1, a_1, r_2 \dots, s_T)$

$G \leftarrow$ retorno total del episodio

$\rho \leftarrow 1$

Para t desde 0 hasta $T-1$:

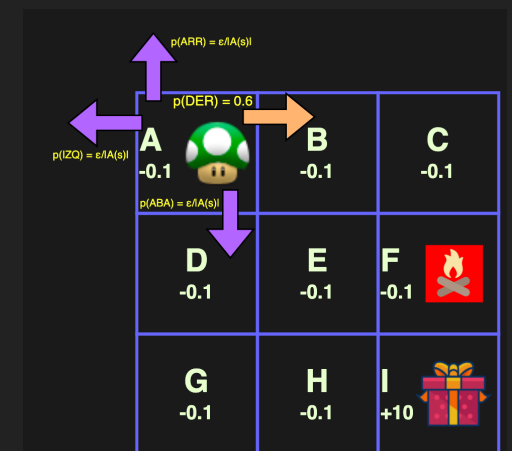
$\rho \leftarrow \rho * \pi(a_t | s_t) / b(a_t | s_t)$

$V(s_0) \leftarrow V(s_0) + \rho * G$

$N(s_0) \leftarrow N(s_0) + 1$

Para cada s :

$V(s) \leftarrow V(s) / N(s)$



MUESTREO POR IMPORTANCIA (IMPORTANCE SAMPLING – IS)

- Muestreo por importancia **ponderado**. (pseudocódigo)

Para cada episodio:

Generar episodio: $(s_0, a_0, r_1, s_1, a_1, \dots, s_T)$

$G \leftarrow$ retorno total del episodio

$\rho \leftarrow 1$

Para t desde 0 hasta $T-1$:

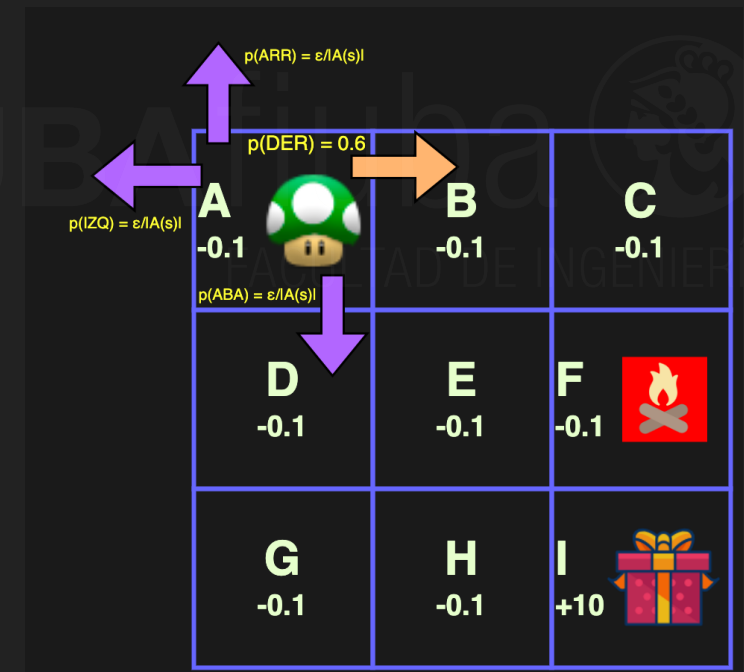
$\rho \leftarrow \rho * \pi(a_t | s_t) / b(a_t | s_t)$

$W(s_0) \leftarrow W(s_0) + \rho * G$

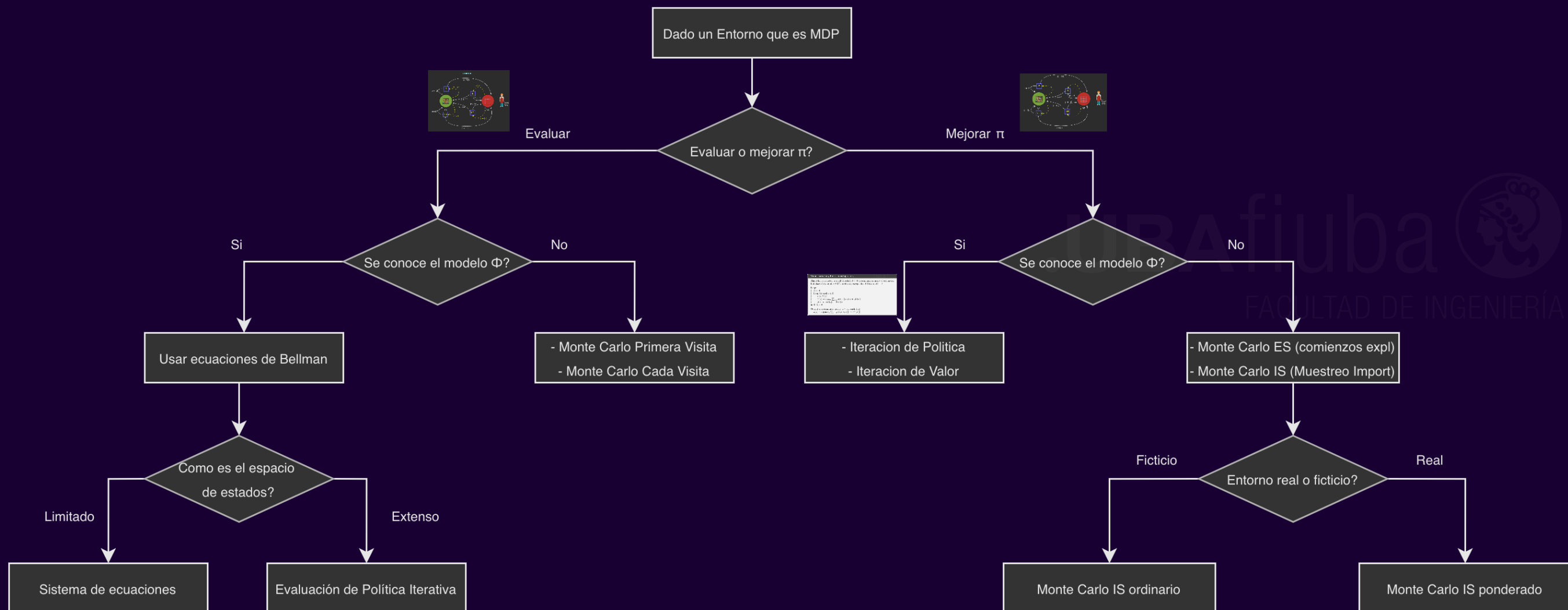
$C(s_0) \leftarrow C(s_0) + \rho$

Para cada s :

$V(s) \leftarrow W(s) / C(s)$



RESUMEN



REFERENCIAS BIBLIOGRÁFICAS Y WEB (I)

- ▶ R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.